Ch14. 데이터 저장

컨테이너 안에 저장한 데이터는 해당 컨테이너가 삭제되었을 때 모두 사라진다. 컨테이너가 사라지더라도 데이터를 보존하도록 컨테이너 외부에 데이터를 저장하는 방법을 알아본다.

- 볼륨 관련 필드 중 .spec.container.volumeMounts.mountPropagation이 있다.
  - 파드 하나 안에 있는 컨테이너끼리 또는 같은 노드에 실행된 파드들끼리 볼륨을 공유해서 사용하지를 설정
  - 필드 값은 다음 3가지를 사용
    - **None**: 볼륨을 마운트했으면, 호스트에서 볼륨에 해당하는 디렉터리 하위에 마운트한 다른 마운트들은 볼 수 없다. 컨테이너가 만들어 놓은 마운트를 호스트에서 볼 수도 없다. 기본 필드 값
    - **HostToContainer**: 볼륨을 마운트했으면 호스트에서 해당 볼륨 하위에 마운트된 다른 디렉터리들도 해당 볼륨에서 볼 수 있도록 한다.
    - **Bidirectiona**l: 볼륨을 마운트했으면 HostToContainer처럼 하위에 마운트된 디렉터리도 볼 수 있고, 호스트 안 다른 모든 컨테이너나 파드에서 같은 볼륨을 사용할 수 있다.

---

1. 볼륨

1.1 emptyDir

emptyDir

- 파드가 실행되는 호스트의 디스크를 실행되는 **호스트의 디스크를 임시로 컨테이너에 볼륨으로 할당해서 사용하는 방법**
- **파드가 사라지면 emptyDir에 할당해서 사용했던 볼륨의 데이터도 함께 사라진다.**
- **주로 메모리와 디스크를 함께 이용하는 대용량 데이터 계산에 사용**
- 과학 연산처럼 오랜 연산 시간이 걸릴 때 중간 데이터 저장용으로 디스크가 필요할 때 사용. 연산 중 컨테이너에 문제가 발생해서 재시작되더라도 파드는 살아 있으므로 emptyDir에 저장해둔 데이터를 계속 사용할 수 있다.
- emptyDir의 설정 예(volume/volume-emptyDir.yaml)

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: kubernetes-simple-pod
spec:
  containers:
  - name: kubernetes-simple-pod
    image: arisu1000/simple-container-app:latest
    volumeMounts:
    - mountPath: /emptydir
      name: emptydir-vol
    # ports:
    # - containerPort: 8080
  volumes:
  - name: emptydir-vol
    emptyDir: {}
```

1.2 hostPath

hostPath

- 파드가 실행된 **호스트의 파일이나 디렉터리를 파드에 마운트한다.**
- emptyDir이 임시 디렉터리에 마운트하는 것이라면 hostPath는 호스트에 있는 실제 파일이나 디렉터리를 마운트한다.
- 또한 **emptyDir이 단순히 컨테이너를 재시작했을 때 데이터를 보존하는 역할이라면 hostPath는 파드를 재시작했을 때도 호스트에 데이터가 남는다.**
- hostPath 설정 예(volume/volume-hostpath.yaml)

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: kubernetes-hostpath-pod
spec:
  containers:
  - name: kubernetes-hostpath-pod
    image: arisu1000/simple-container-app:latest
    volumeMounts:
```

```
      - mountPath: /test-volume
        name: hostpath-vol
      ports:
      - containerPort: 8080
    volumes:
    - name: hostpath-vol
      hostPath:
        path: /tmp
        type: Directory
```

[.spec.volumes[].hostpath.type 필드 값 종류]

필드 값	내용
설정하지 않음	hostPath 볼륨을 마운트하기 전 아무것도 확인하지 않는다.
DirectoryOrCreate	설정한 경로에 디렉터리가 없으면 퍼미션 755인 빈 디렉터리를 만든다.
Directory	설정한 경로에 디렉터리가 존재해야 한다. 없으면 파드는 ContainerCreating 상태로 남는다.
FileOrCreate	설정한 경로에 파일이 없으면 퍼미션 644인 빈 파일을 만든다.
File	설정한 경로에 파일이 있는지 확인한다. 파일이 없으면 파드 생성이 안된다.
Socket	설정한 경로에 유닉스 소켓 파일이 있어야 한다.
CharDevice	설정한 경로에 Character 디바이스가 있는지 확인한다.
BlockDevice	설정한 경로에 Block 디바이스가 있는지 확인한다.

1.3 nfs

nfs(network file system)

- 기존에 사용하는 NFS 서버를 이용해서 파드에 마운트하는 것
- NFS 클라이언트 역할
- 여러 개 파드에서 볼륨 하나를 공유해 읽기/쓰기를 동시에 할 때도 사용
- NFS 서버를 설정하는 예(volume/volume-nfsserver.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nfs-server
  labels:
```

```yaml
    app: nfs-server
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nfs-server
  template:
    metadata:
      labels:
        app: nfs-server
    spec:
      containers:
      - name: nfs-server
        image: arisu1000/nfs-server:latest
        ports:
        - name: nfs
          containerPort: 2049
        - name: mountd
          containerPort: 20048
        - name: rpcbind
          containerPort: 111
        securityContext:
          privileged: true
        volumeMounts:
        - mountPath: /exports
          name: hostpath-vol
      volumes:
      - name: hostpath-vol
        hostPath:
          path: /tmp
          type: Directory
```

- nfskubectl apply -f volume-nfsserver.yaml
- kubectl get pods -o wide -l app=nfs=server
- NFS 서버에 접속할 클라이언트 앱 컨테이너 설정(volume/volume-nfsapp.yaml)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kubernetes-nfsapp-pod
  labels:
    app: nfs-client
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nfs-client
  template:
    metadata:
      labels:
        app: nfs-client
    spec:
      containers:
      - name: kubernetes-nfsapp-pod
        image: arisu1000/simple-container-app:latest
        volumeMounts:
        - mountPath: /test-nfs
          name: nfs-vol
        ports:
        - containerPort: 8080
      volumes:
      - name: nfs-vol
        nfs:
          server: 10.1.6.126    # 여기에 nfs-server pod의 ip
를 넣습니다.
          path: /
```

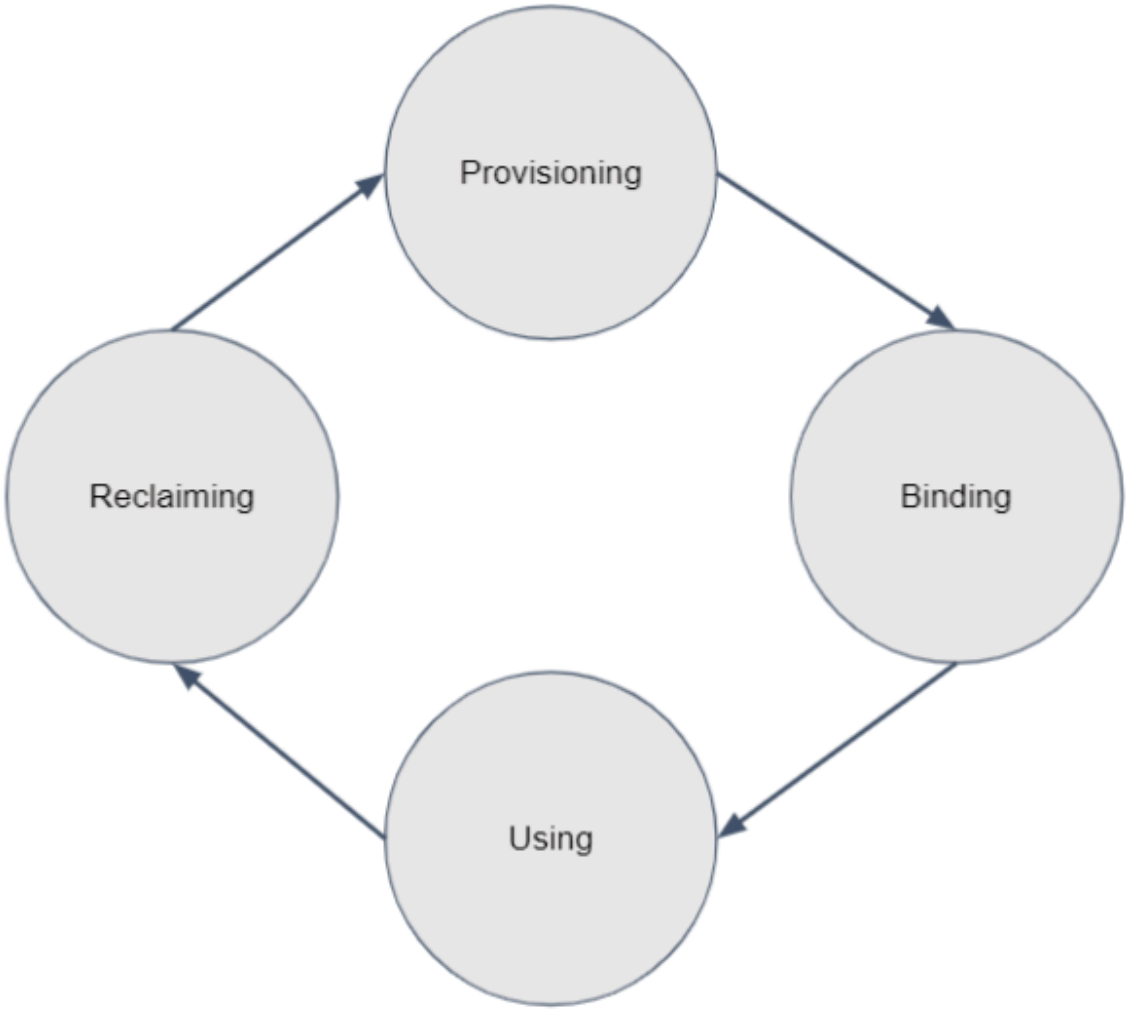
---

2. 퍼시스턴트 볼륨과 퍼시스턴트 볼륨 클레임

쿠버네티스에서 **볼륨을 사용하는 구조는 PV라는 PersistentVolume과 PVC라는 PersistentVolumeClaim 2개로 분리되어 있다.**

- **PV는 볼륨 자체**를 뜻한다. 클러스터 안에서 자원을 다룬다. 파드하고는 별개로 관리되고 별도의 생명 주기가 있다.
- **PVC는 사용자가 PV에 하는 요청**이다. 사용하고 싶은 용량은 얼마인지, 읽기/쓰기는 어떤 모드로 설정하고 싶은지 등을 정해서 요청한다.
- 쿠버네티스는 볼륨을 파드에 직접 할당하는 방식이 아니라 중간에 PVC를 두어 파드와 파드가 사용할 스토리지를 분리했다.
- 다양한 스토리지를 PV로 사용할 수 있지만 파드에 직접 연결하는 것이 아니라 PVC를 거쳐서 사용하므로 파드는 어떤 스토리지를 사용하는지 신경쓰지 않아도 된다.

[PV와 PVC의 생명 주기]



2.1 Provisioning

- **PV를 만드는 단계**

- Provisioning 방법에는 2가지가 있다. PV를 미리 만들어 두고 사용하는 **정적 방법**과 요청이 있을 때마다 PV를 만드는 **동적 방법**이다.

2.2 Binding

- Provisioning으로 만든 **PV를 PVC와 연결하는 단계**

2.3 Using

- **PVC는 파드에 설정되고 파드는 PVC를 볼륨으로 인식해서 사용**
- 할당된 PVC는 파드를 유지하는 동안 계속 사용하며 시스템에서 임의로 삭제할 수 없다. 이 기능은 '**사용 중인 스토리지 오브젝트 보호(Storage Object in Use Protection)**'라고 한다.

2.4 Reclaiming

- **사용이 끝난 PVC는 삭제되고 PVC를 사용하던 PV를 초기화(reclaim)하는 과정을 거친다.** 이를 반환이라 한다.
- **초기화 정책**은 Retain, Delete, Recycle이 있다.
  - **Retain**: PV를 그대로 보존
  - **Delete**: PV를 삭제하고 연결된 외부 스토리지 쪽의 볼륨도 삭제
  - **Recycle**: PV의 데이터들을 삭제하고 다시 새로운 PVC에서 PV를 사용할 수 있도록 한다.

---

3. Persistent Volume Template

- 퍼시스턴트 볼륨 설정의 예(volume/pv-hostpath.yaml)
  - .spec.capacity.storage
  - .spec.volueMode: 기본 필드 값은 Filesystem으로 볼륨을 파일 시스템 형식으로 설정해서 사용 추가로 raw라는 필드 값을 설정할 수 있다.
  - .spec.**volumeModes** 필드는 볼륨의 읽기/쓰기 옵션을 설정. 필드 값은 3가지
    - **ReadWriteOnce**: 노드 하나에만 볼륨을 읽기/쓰기하도록 마운트할 수 있음
    - **ReadOnlyMany**: 여러 개 노드에서 읽기 전용으로 마운트할 수 있음
    - **ReadWriteMany**: 여러 개 노드에서 읽기/쓰기 가능하도록 마운트할 수 있음

- .spec.storageClassName: 스토리지 클래스를 설정하는 필드. 특정 스토리지 클래스가 있는 PV는 해당 스토리지 클래스에 맞는 PVC와만 연결. PV에 .spec.storageClassName 필드 설정이 없으면 .spec.storageClassName 필드 설정이 없는 PVC와만 연결
- .spec.persistentVolumeReclaimPolicy 필드는 PV가 해제되었을 때의 초기화 옵션을 설정. **Retain, Recycle, Delete** 정책 중 하나를 설정
- .spec.hostPath 필드는 해당 PV의 볼륨 플러그인을 명시한다.
- 퍼시스턴트 볼륨 설정의 예(volume/pv-hostpath.yaml)

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-hostpath
spec:
  capacity:
    storage: 2Gi
  volumeMode: Filesystem
  accessModes:
  - ReadWriteOnce
  storageClassName: manual
  persistentVolumeReclaimPolicy: Delete
  hostPath:
    path: /tmp/k8s-pv
```

- kubectl apply -f pv-hostpath.yaml
- kubectl get pv
  - STATUS 항목이 Available이면 정상적으로 설정된 것

```
$ kubectl get pv
NAME          CAPACITY   ACCESS MODES   RECLAIM POLICY   ST
ATUS      CLAIM   STORAGECLASS   REASON   AGE
pv-hostpath   2Gi        RWO            Delete           Av
ailable           manual                  7s
```

---

4. Persistent Volume Claim Template

- 퍼시스턴트 볼륨 클레임의 설정 예(volume/pvc-hostpath.yaml)
  - .spec.resources.requests.storage 필드: 자원을 얼마나 사용할 것인지 요청
    - 만약에 필드 값으로 PV의 용량 이상을 설정하면 사용할 수 있는 PV가 없으므로 PVC는 생성할 수 없는 Pending 상태가 된다.

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: pvc-hostpath
spec:
  accessModes:
  - ReadWriteOnce
  volumeMode: Filesystem
  resources:
    requests:
      storage: 1Gi
  storageClassName: manual
```

- kubectl apply -f pvc-hostpath.yaml
- kubectl get pvc
  - STATUS 항목은 Bound
  - VOLUME 항목은 pv-hostpath

```
$ kubectl get pvc
NAME           STATUS   VOLUME        CAPACITY   ACCESS MOD
ES   STORAGECLASS   AGE
pvc-hostpath   Bound    pv-hostpath   2Gi        RWO
manual         8s
```

- kubectl get pv
  - PV가 PVC와 연결되어서 STATUS 항목이 Bound

```
$ kubectl get pv
NAME          CAPACITY   ACCESS MODES   RECLAIM POLICY   ST
```

```
ATUS   CLAIM                    STORAGECLASS   REASON   AGE
pv-hostpath   2Gi        RWO            Delete           Bo
und    default/pvc-hostpath   manual                  47h
```

---

5. 레이블로 PVC와 PV 연결하기

- **Persistent Volume은 쿠버네티스 안에서 사용되는 자원이고 PVC은 해당 자원을 사용하겠다고 요청하는 것**이므로 **파드와 서비스를 연결할 때처럼 레이블을 사용할 수 있다.**
- 퍼시스턴트 볼륨에 레이블 관련 설정을 추가한 예(volume/pv-hostpath-label.yaml)
  - .metadata.labels.location 필드에 local이라는 값을 설정해서 레이블을 추가

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-hostpath-label
  labels:
    location: local
spec:
  capacity:
    storage: 2Gi
  volumeMode: Filesystem
  accessModes:
  - ReadWriteOnce
  storageClassName: manual
  persistentVolumeReclaimPolicy: Delete
  hostPath:
    path: /tmp/k8s-pv
```

- 퍼시스턴트 볼륨 클레임에 레이블 관련 설정을 추가한 예(volume/pvc-hostpath-label.yaml)
  - .spec.selector.matchLabels.location 필드에 앞에서 사용했던 필드 값인 local을 설정

```yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
```

```
  name: pvc-hostpath-label
spec:
  accessModes:
  - ReadWriteOnce
  volumeMode: Filesystem
  resources:
    requests:
      storage: 1Gi
  storageClassName: manual
  selector:
    matchLabels:
      location: local
```

- kubectl apply -f pv-hostpath-label.yaml
- kubectl apply -f pvc-hostpath-label.yaml

---

6. 파드에서 PVC를 볼륨으로 사용하기

- PVC를 실제 파드에서 사용하는 설정 예(volume/deployment-pvc.yaml)
  - .spec.template.spec.volumes[].name 필드 값은 사용할 볼륨을 설정
  - 준비한 볼륨을 실제 컨테이너에 연결하는 것은 .spec.template.spec.containers[].volumeMounts[]의 하위 필드다.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kubernetes-simple-app
  labels:
    app: kubernetes-simple-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: kubernetes-simple-app
  template:
    metadata:
```

```yaml
      labels:
        app: kubernetes-simple-app
    spec:
      containers:
      - name: kubernetes-simple-app
        image: arisu1000/simple-container-app:latest
        ports:
        - containerPort: 8080
        imagePullPolicy: Always
        volumeMounts:
        - mountPath: "/tmp"
          name: myvolume
      volumes:
      - name: myvolume
        persistentVolumeClaim:
          claimName: pvc-hostpath
```

- kubectl apply -f deployment-pvc.yaml